# Multi-target IC vs rho in wall time (2026-11-12)

# Multi-target IC vs rho in wall time: the IC is faster on 2^24 groups and slower on 2^32 groups

Protocol: `PROTOCOL.md` (frozen in a992ed6c80 before data). **1836 executions (36 configurations x 17 keys x 3
interleaved repeats), 0 errors, every target of every run verified**, sequential and pinned to one CPU of the
session host (4-vCPU cloud VM, Intel Xeon @ 2.10 GHz). Data: `results/holdout/runs.jsonl`, `opcost.jsonl`,
`analysis.txt`, `summary.json`. Every number here is [pcc:unverified] until the claim is signed. Independent
validation: `validation/VALIDATOR_REPORT.md` (counters replay exactly for all 1836 executions, 319k recovered logs
checked with independent EC code, every number recomputes, an independent pinned rerun of 8 repeats reproduces all 9
decisions); its amendments are applied here. The configurations reuse the 12 v5 curves (not fresh) with a new seed.

**Validated headline (the validator's wording):** On one CPU of a 4-vCPU cloud VM (Intel Xeon @ 2.10 GHz; not an isolated lab node, so the seconds are diagnostic), in th

**Host caveat.** The lab's default route (Iso Lab on EKS, L2 isolation) was unavailable: this session's AWS
identity is a placeholder that AWS rejects. The seconds below are from a shared cloud VM without a hardware PMU
and are diagnostic, not lab-admitted; the study's own controls (pinning, interleaved repeats, A/A, a selection-bias
control) are reported with them.

## Result (solve-phase wall time, median of 4 configurations per cell, bootstrap 95%)

| bits | T | ic/rho solve [95%] | ic/rho incl. verify [95%] | ic/bsgs solve | A/A rep0/rep1 (rho side) [95%] | fixed-key ic/rho solve [95%] | winners ic / rho |
|---|---|---|---|---|---|---|---|
| 24 | 1024 | 0.778 [0.765, 0.922] cheaper | 0.818 [0.791, 0.943] | 1.537 | 0.863 [0.667, 1.145] | 0.825 [0.637, 0.961] | {'ic5w': 2, 'ic5': 2} / {'rho2xks 0.5': 2, 'rho
| 24 | 4096 | 0.792 [0.753, 0.821] cheaper | 0.852 [0.816, 0.878] | 1.326 | 0.985 [0.832, 1.358] | 0.818 [0.731, 0.847] | {'ic5': 3, 'ic5w': 1} / {'rhodpu 0.0': 3, 'rho2
| 24 | 16384 | 0.829 [0.811, 0.872] cheaper | 0.922 [0.885, 0.944] | 1.438 | 1.113 [0.975, 1.421] | 0.752 [0.655, 1.211] | {'ic5': 1, 'ic5w': 3} / {'rhodpu 0.0': 3, 'rho
| 28 | 1024 | 0.932 [0.869, 0.967] cheaper | 0.929 [0.858, 0.966] | 1.374 | 0.992 [0.830, 1.042] | 0.975 [0.884, 1.137] | {'ic5': 3, 'ic5w': 1} / {'rhodpu 0.0': 4} |
| 28 | 4096 | 1.053 [0.987, 1.101] unranked | 1.046 [0.989, 1.094] | 1.544 | 0.990 [0.809, 1.001] | 1.031 [0.967, 1.143] | {'ic5w': 2, 'ic5': 2} / {'rhodpu 0.0': 4} |
| 28 | 16384 | 1.007 [0.954, 1.193] unranked | 1.025 [0.976, 1.159] | 1.443 | 0.890 [0.725, 1.150] | 1.017 [0.940, 1.077] | {'ic5w': 3, 'ic5': 1} / {'rhodpu 0.0': 4} |
| 32 | 1024 | 1.114 [1.049, 1.211] dearer | 1.115 [1.048, 1.203] | 1.456 | 1.034 [0.894, 1.344] | 1.096 [0.972, 1.351] | {'ic5': 2, 'ic5w': 2} / {'rhodpu 0.0': 4} |
| 32 | 4096 | 1.203 [1.116, 1.303] dearer | 1.195 [1.105, 1.294] | 1.421 | 0.933 [0.812, 1.110] | 1.180 [1.052, 1.305] | {'ic5': 2, 'ic5w': 2} / {'rhodpu 0.0': 4} |
| 32 | 16384 | 1.289 [1.269, 1.319] dearer | 1.268 [1.252, 1.300] | 1.590 | 0.975 [0.846, 1.056] | 1.284 [1.218, 1.374] | {'ic5': 4} / {'rhodpu 0.0': 3, 'rhodpu 0.25': 1

- **2^24 groups: the IC is faster in all three cells (0.78-0.83x)** [pcc:unverified]; 2^28: faster at T = 1024 (0.93x), unranked at
  T = 4096 and 16384; **2^32: slower in all three cells (1.11-1.29x)**. The IC's relative speed falls as the group
  grows, and the trend runs against the IC toward cryptographic sizes.
- With the shared verification pass included the ratios are 0.82-1.27x, the same pattern.
- Each cell decision rests on 4 configurations, and with n = 4 the bootstrap interval of the median equals their
  range, so a decision is a 4-of-4 sign rule (p = 0.125 per cell). The strong evidence is at the size level: 12 of 12
  configurations below 1 at 2^24 and 12 of 12 above 1 at 2^32.
- Selection-bias control (each side's key fixed on repeat 0, scored on repeats 1-2): 0.75-1.28x, the same direction
  by size, but unranked in 3 of 9 cells (24 b T = 16384, 28 b T = 1024, 32 b T = 1024).
- 28 b T = 1024 (0.93x) is fragile: unranked under the control, 0.98x [0.959, 0.999] in the validator's pooled rerun.
- Independent reproduction (validator, same host and binary, pinned, producer's keys): 6 of 9 decisions in a
  3-repeat round, 8 of 9 in a 5-repeat round, 9 of 9 pooled (2^24 0.77-0.84x, 2^32 1.11-1.22x); per-configuration
  ratios differ between sessions by a median of 5% (max 22%).
- A/A: the rho-side repeat-0/repeat-1 ratio's interval contains 1 in all 9 cells (widest 24 b T = 1024: 0.863
  [0.667, 1.145]); over all keys the median |log(repeat/median)| is
  0.028 (2.8%) and the 95th percentile 0.28 (dominated by the short 24-bit runs). The 2^32 differences (11-29%)
  are well above the median noise, and their intervals exclude 1.
- Distance to BSGS: the IC is 1.33-1.59x slower than the fastest paired BSGS in every cell.
- Fastest arms: rho side `rhodpu` in 30 of 36 configurations (kappa 0: 29, kappa 0.25: 1), `rho2xks` (kappa 0.25-0.5) in 6; IC side
  `ic5` 20, `ic5w` 16. Retention (`ic5ks`, `ic5rw`, the hash-heavy variants) never wins in time.

## Measured operation prices (`opcost`, before and after, curve 0 of each size)

- One affine addition (1 GAE): 214-294 ns. Inversion: I = 23-31 M (the section 5a band's c_I = 30 is the matching
  price; c_I = 3 and 10 are far from this implementation).
- A canonicalisation plus one hash of x (two counted native operations) costs 0.005-0.009 GAE, so one counted native
  operation 0.0025-0.0047 GAE; a step index 0.002-0.006 GAE; both at the loop baseline: **in practice h_native is
  about 0**.
- A std HashMap lookup (SipHash, the harness's table): 0.04-0.15 GAE at 2^10-2^16 entries, 0.11-0.42 at 2^18-2^20,
  0.27-0.85 at 2^22-2^24 (cache misses). A fast multiplicative hasher is 1.2-6.1x cheaper (`opcost.jsonl`). Lookups and native operations are timed
  as independent-iteration loops (independent probes; the hit/miss mix varies with table size; single loop segments
  can be up to about 4x off), so these are prices of this microbenchmark, not of the solvers' access patterns.

## Answer to the v5 question (interpretation)

A price model from the measured lookup prices predicts 1.14-1.22x at 2^32 (consistent with the measurement) but
0.85-1.02x at 2^24 against the measured 0.78-0.83x, so the model does not explain the small-group advantage fully
(`validation/price_model.json`).

v5 priced native work and hash-table operations at the same h. On this host they differ by one to two orders of
magnitude: native work is nearly free (h_native about 0) while lookups cost h_hash about 0.05-0.85 depending on
table size. That combination favours the distinguished-point rho, which does many native operations and few
lookups, and it is the rho that wins in time (30 of 36 configurations), although it won only 4 of 198
configurations in v5's counted model. At small groups the IC still wins (its tables are small and cache-resident, and its per-target work is
lower); at 2^32 its table and trials are larger, lookups become expensive (that this is cache misses is an
interpretation; there is no PMU on this host), and the distinguished-point rho is faster by
11-29% [pcc:unverified]. So in this implementation on this host the multi-target IC does not beat Pollard rho at the largest size
tested, and the trend is against it as the group grows.

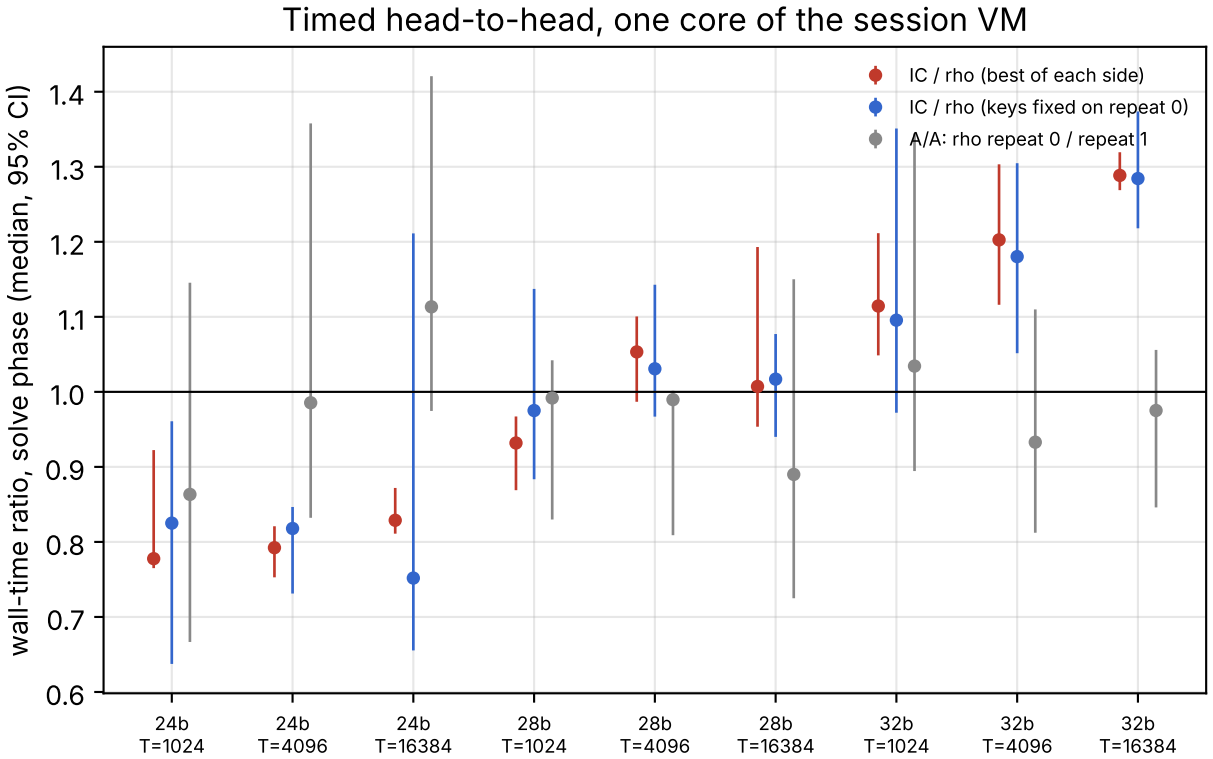
## Pre-registered predictions vs outcome

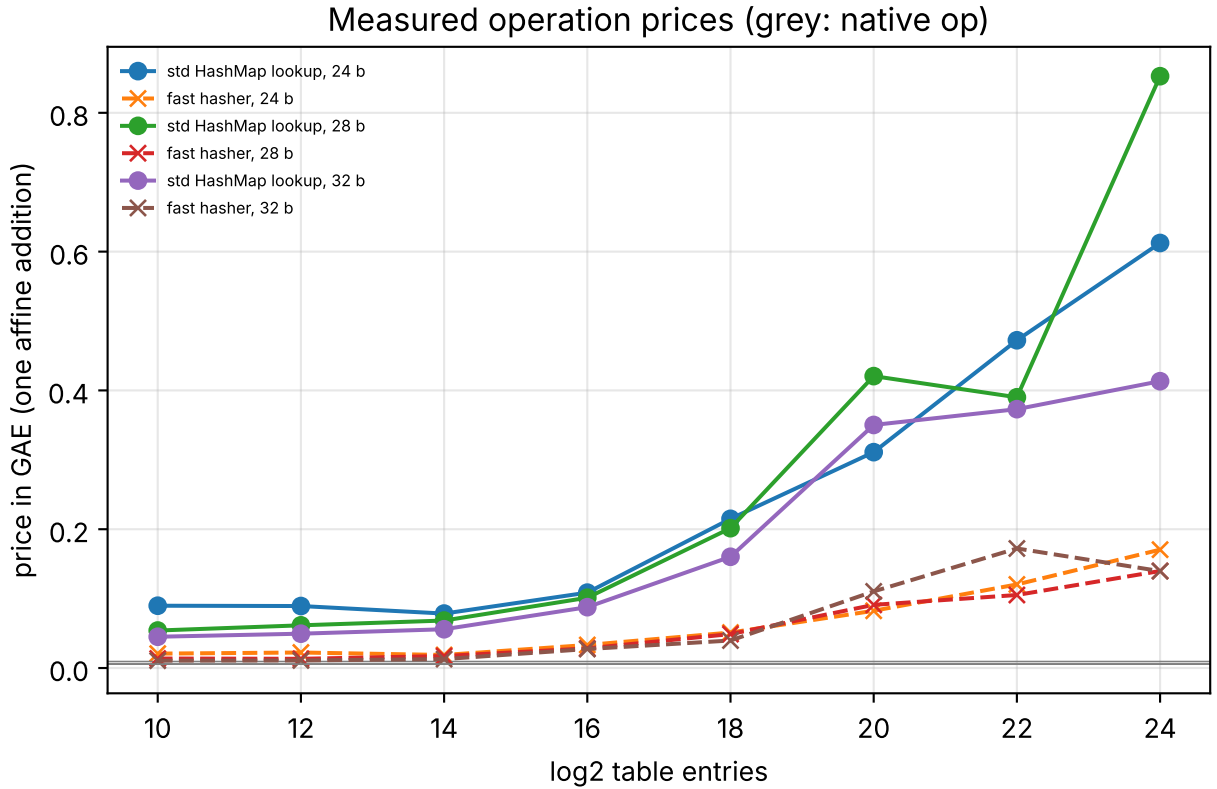
| prediction | outcome |
|---|---|
| I/M about 23-28; native about 0.005-0.01 GAE; std lookup 0.05-0.1 at 2^10-2^16, 0.4-0.6 at 2^22-2^24; fast hasher 2-5x cheaper | mostly holds (I/M 23-31; canonicalisat
| IC faster than rho in most cells (0.8-0.95), most clearly at large tables (32 b, large T) | wrong in direction of size: faster at 24 b (0.78-0.83), slower at 32 b (1.1
| IC slower than BSGS everywhere | holds (1.33-1.59x) |

## Limits

One shared cloud VM (possible steal time; no PMU), not an isolated L2 node, and one CPU of it; one implementation
(std HashMap with SipHash, counting cells in the field layer, u64 field arithmetic with u128 reductions); 4 curves
per size and one seed (4 configurations per cell); the comparison is per target in a batch of T on one curve,
memory unbounded; it says nothing about single-target ECDLP, about other implementations or hardware, or about
ECC2K-130 / m = 83. The smoke run before the freeze (W24-0, seed 43, T = 1024, one of the holdout configurations)

was not preserved. `runs.jsonl` goes to the artifact store when the PR opens (`runs.json` keeps the table in Git).
A rerun on EKS L2 with the same `run.py` and binary is the follow-up.


Timed head-to-head, one core of the session VM
wall-time ratio, solve phase (median, 95% CI)
IC / rho (best of each side)
IC / rho (keys fixed on repeat 0)
A/A: rho repeat 0 / repeat 1
0.6
0.7
0.8
0.9
1.0
1.1
1.2
1.3
1.4
24b T=1024
24b T=4096
24b T=16384
28b T=1024
28b T=4096
28b T=16384
32b T=1024
32b T=4096
32b T=16384


Measured operation prices (grey: native op)
price in GAE (one affine addition)
std HashMap lookup, 24 b
fast hasher, 24 b
std HashMap lookup, 28 b
fast hasher, 28 b
std HashMap lookup, 32 b
fast hasher, 32 b
0.0
0.2
0.4
0.6
0.8
10
12
14
16
18
20
22
24
log2 table entries